Corrected Document Output

Generated on: 2025-06-10 05:54:03

Chosen Field and MotivationThe project focuses on fruit detection and classification using computer vision. Fruit classification plays a crucial role in agricultural automation, quality control, and smart retail. Automating the identification of different fruits can enhance efficiency in sorting, packaging, and quality assurance processes, reducing the need for manual labor and minimizing errors. Dataset Information Name: Fruits Dataset Source: Kaggle - Fruits262 Number of Classes: 6 (Apple, Banana, Grape, Kiwi, Orange, Pineapple) Number of Images: 6000 images total Why This Dataset? Choosing a larger, pre-segmented dataset would've limited our ability to get hands-on experience with segmentation techniques. We specifically wanted to challenge ourselves by handling the full process-from raw images to segmentation and classification-so this dataset offered the perfect balance of quality, size, and opportunity to learn. Detailed explanation of each step in the pipeline Step 1: Data Loading and Splitting The dataset was loaded from a remote directory where each fruit class had its own labeled subfolder. Images were read using OpenCV (cv2), and class labels were inferred from the folder names. Each image and its corresponding label were stored in a list or dataframe, ensuring structured access. To ensure reliable training and evaluation, the dataset was split into training and validation sets using train_test_split, with stratification applied to preserve class distribution. Step 2: Preprocessing & Image Enhancement●■ All images were resized to a consistent 128×128 pixel resolution to match input requirements across all models. As OpenCV loads images in BGR format, all images were converted to RGB to maintain color consistency. Pixel values were normalized to a [0, 1] range to speed up training and stabilize model performance. Any corrupted or unreadable images were automatically skipped during preprocessing to avoid disrupting the pipeline.

Step 3: Dataset Preparation Class labels were converted into integers using LabelEncoder. One-hot encoding was applied to convert labels into a format suitable for multi-class classification. Final training and validation datasets were reshaped as needed for neural network input. Step 4: Feature Extraction (for Classical Models) For Random Forest (RF) and Support Vector Machine (SVM) models, handcrafted features were extracted: Color features: Mean values of R, G, and B channels. Shape features: Area, perimeter, aspect ratio from segmented fruits. Texture features by using grayscale histograms or GLCM (Gray Level Co-occurrence Matrix). These numerical features were compiled into a feature matrix used for training classical models. Step 5: Feature Extraction (for CNN Models) CNN models automatically

learned features through: Convolutional layers, which detect patterns such as edges, colors, and shapes. Models trained on segmented data extracted features only from fruit regions, improving learning efficiency and reducing background noise. Step 6.0x2x2.0 Segmentation (for One CNN Variant) GPU GPU Settings (for a CNN model variant, a background removal was performed using HSV color segmentation. This isolated the fruit from the background, helping the model focus on relevant features. Morphological operations such as erosion and dilation were applied to clean up masks. Step 7: Model Building and Modeling●■ A custom Convolutional Neural Network (CNN) was designed using TensorFlow (TensorFlow/Keras: The Neural Network) as the primary layer.●■■■ convolutional layers for feature extraction. MaxPooling layers to reduce dimensionality and overfitting. ■■ Dense layers for final classification. ReLU activations for non-linearity and Softmax for output. Classical models like RF and SVM were built using scikit-learn on the handcrafted feature dataset. MobileNetV2 was integrated as the base model for feature extraction, with custom dense layers for classification, using transfer learning to leverage pre-trained weights from ImageNet.

Step 8: Model Training CNN models and MobileNetV2 were trained using categorical crossentropy as the loss function and the Adam optimizer. Latency, batch size and number of epochs were selected based on convergence trends and available compute power. RF and SVM models were trained using standard scikit-learn pipelines with cross-validation. Step 9: Evaluation and Visualization of the Model●■ Model performance was monitored using accuracy and loss plots across training epochs. Confusion matrices were generated to observe model confusion among classes. Sample predictions were visualized alongside actual labels to qualitatively assess model behavior. Input and Output of Each Step Step Input Output Image Loader Raw fruit images Resized and normalized image arrays Label encoder Fruit names (strings) Encoded labels and one-hot vectors Trained CNN model CNN Model Training Training data (images + labels) Evaluation Test data Accuracy, loss, confusion matrix, visual plots Techniques and justification for each technique's selection Convolutional neural network (CNN): Effective in image classification due to spatial feature learning. ReLU Activation: Prevents vanishing gradients and speeds up convergence. Softmax Output: Suits multi-class classification problems. ●■ Adam Optimizer: Adaptive learning rate helps with faster and more stable convergence.●■ Argmax: Used during inference to select the class with the highest predicted probability from the Softmax output. Chosen for its ability to enhance the distinction of object boundaries in images, which is critical for tasks like object recognition and classification.

Ly, Visual Examples of Detection, Output Preferences, Package Settings MobileNetV2.

CNN.

CNN with Segmentation.

Evaluation metrics and discussion topics:●■ CNN

CNN with segmentation.

MobileNet.

COMPARISONS.

Limitations or Challenges Faced with:●■ Class Imbalance: Some classes have more examples than others. Hardware Limitations: Training time and batch sizes limited by GPU availability. Visual Similarity: Certain fruits (like apple vs. pear) can be hard to differentiate. ●■ Insufficient Image Diversity: The dataset may not provide enough variety of images per class, which can affect model generalization. Some class names may be ambiguous or inconsistent, leading to potential confusion during training. References:●■ Kaggle●■ Fruit-262 Dataset: https://www.kaggle.com/datasets/kritikseth/fruit-262.html●■ TensorFlow Documentation: http://tensorflow.org/api_docs/TensorFlow-262●■ Keras API Guide:https://keras.io/api/